Fiche patterns – Arbres généraux

`Tree (key, children, nbchildren)` · `TreeAsBin (key, child, sibling)` · `Queue (enqueue, dequeue, isempty)`

Étape 1 – BFS ou DFS ? Un seul mot décide

La règle

Le mot **niveau** (ou **largeur**) est dans l'énoncé ⇒ **BFS**.

Sinon ⇒ **DFS** (c'est le choix **par défaut**).

Exception : le sujet **impose** le parcours (« en profondeur ») ⇒ on obéit.

Mots qui veulent dire BFS

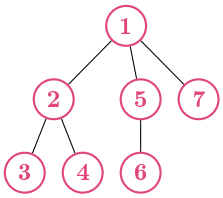
- **niveau**, par niveau, chaque niveau
- **largeur**
- même **profondeur que x** (sur le même niveau)
- le **plus proche de la racine**
- le premier rencontré « en largeur »

Mots qui veulent dire DFS

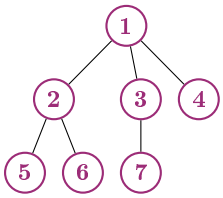
- **fils**, **père**, **sous-arbre**
- **chemin**, **feuille**
- **hauteur**, taille, somme, moyenne
- profondeur **d'un nœud** ou **des feuilles**
- « parcours profondeur » imposé

Énoncé		Mot repère	Parcours
Chaque **niveau** a plus de nœuds que le précédent		niveau	BFS
Nombre max de nœuds sur un même **niveau**		niveau	BFS
Clés après x sur le même **niveau**		niveau	BFS
Nœud de clé x le **plus proche de la racine**		plus proche	BFS
Clé de chaque nœud = somme des clés de ses **fils**		fils	DFS
Existe-t-il un **chemin** racine → feuille de somme s ?		chemin	DFS
Moyenne de chaque **sous-arbre** ≤ clé du nœud		sous-arbre	DFS
Profondeur moyenne des **feuilles**	(piège)	feuilles	DFS

Pourquoi ? Sur le même arbre, voici l'ordre de visite :



DFS : descend d'abord



BFS : niveau par niveau

- Le **BFS** donne tous les nœuds d'un niveau **à la suite** → on peut compter / comparer par niveau et **s'arrêter tôt**.
- Le **DFS** mélange les niveaux, mais il voit chaque nœud **avec ses fils** → idéal pour père/fils, sous-arbres, chemins.

Étape 2 – Les 4 questions avant d'écrire une ligne

1. **BFS ou DFS ?** → le mot repère ci-dessus.
2. **Que renvoie une feuille ?** → fixe le cas de base et la valeur de départ.
3. **Comment je combine les fils ?** → somme, `max`, « tous vrais », « au moins un vrai ».
4. **Puis-je m'arrêter avant la fin ?** → si oui : `while C and ...` ou `return` dans la boucle.

DFS 1 – Accumuler (taille, feuilles, hauteur)

Comment lire les squelettes

Les mots **EN MAJUSCULES** (`DEPART`, `COMBINE`, `FIN`, `LOCAL_OK`, `TEST`) ne sont **pas** du Python : ce sont des **trous à remplir** selon l'exo. Les commentaires disent quoi mettre.

On appelle la fonction sur **chaque fils**, on combine, et on fait `return` **après** la boucle.

`Tree`

```
def f(T):
    acc = DEPART          # ce que vaut une feuille
    for C in T.children:  # chaque fils C, un par un
        r = f(C)          # résultat du sous-arbre C
        acc = COMBINE(acc, r)  # ex : acc + r
                               #    ou max(acc, r)
    return FIN(acc)       # souvent juste acc
```

`TreeAsBin`

```
def f(B):
    acc = DEPART          # ce que vaut une feuille
    C = B.child           # premier fils
    while C:              # tant qu'il reste un fils
        r = f(C)          # résultat du sous-arbre C
        acc = COMBINE(acc, r)
        C = C.sibling     # fils suivant (obligatoire !)
    return FIN(acc)
```

Fonction	Départ	Combinaison	Fin	`if` feuille ?
taille	`s = 1`	`s += f(C)`	`s`	non
feuilles	`s = 0`	`s += f(C)`	`s`	oui : `return 1`
hauteur (feuille = 0)	`h = -1`	`h = max(h, f(C))`	`h + 1`	non
arité max	`k = m = 0`	`k += 1 ; m = max(m, f(C))`	`max(k, m)`	non

Le `if` feuille ne sert que si une feuille renvoie autre chose que la formule générale avec zéro fils.

DFS 2 – Vérifier une propriété avec arrêt anticipé

On s'arrête au **premier** `False` : c'est ce que l'EPITA note en optimisation.

Sans `return` dans la boucle

```
def check(B):
    if not LOCAL_OK(B):   # B lui-même respecte
        return False      # la règle ? (ex : B.key >= 0)
    C = B.child
    while C and check(C): # on avance tant que
        C = C.sibling     # les fils sont OK
    return C is None      # None : tous vus, tous OK
                          # sinon : un fils a échoué
```

Avec `return` dans la boucle

```
def check(T):
    if not LOCAL_OK(T):   # T lui-même respecte
        return False      # la règle ?
    for C in T.children:
        if not check(C):  # un fils échoue :
            return False  # on arrête tout de suite
    return True           # tous les fils sont OK
```

Variante « il existe » (chemin de somme s) : on s'arrête au **premier** `True`.

```
C = B.child
while C and not f(C):   # on cherche un fils qui réussit
    C = C.sibling
return C is not None    # True : on en a trouvé un
```

DFS 2 bis – « CHAQUE » ou « IL EXISTE » ?

Le mot de l'énoncé choisit la boucle

« **chaque** », « **tous** » ⇒ tous les fils doivent réussir ⇒ on s'arrête au **premier échec**.
« **existe-t-il** », « **au moins un** » ⇒ un seul fils suffit ⇒ on s'arrête au **premier succès**.
Rappel : la condition d'un `while` dit quand on **continue**, pas quand on s'arrête.

CHAQUE / TOUS (check_sum, check_max, heap_min)

Version `while`

```
C = B.child
while C and f(C):      # tant que le fils RÉUSSIT
    C = C.sibling      # on continue
return C is None       # au bout = tous OK
```

Version `if`

```
C = B.child
while C:
    if not f(C):       # un fils échoue
        return False   # tout est faux
    C = C.sibling
return True            # aucun échec
```

IL EXISTE (path_sum, path_increasing, leaf_at_depth)

Version `while`

```
C = B.child
while C and not f(C): # tant que le fils ÉCHOUE
    C = C.sibling     # on continue
return C is not None  # arrêté sur un fils = trouvé
```

Version `if`

```
C = B.child
while C:
    if f(C):           # un fils réussit
        return True    # trouvé
    C = C.sibling
return False           # aucun succès
```

	CHAQUE	IL EXISTE
Condition du `while`	`f(C)`	`not f(C)`
`return` final	`C is None`	`C is not None`
Version `if` : dans la boucle	`if not f(C): return False`	`if f(C): return True`
Version `if` : après la boucle	`return True`	`return False`

Les deux sont **en miroir** : là où l'une met un `not`, l'autre l'enlève, et les `True` / `False` s'échangent.
Le `not` va **soit** dans le `while`, **soit** dans le `return` : jamais les deux, jamais aucun.

Faut-il une fonction auxiliaire ? Écris la phrase d'abord

« Vrai si … dans le sous-arbre B, **sachant que** … » (c'est la docstring exigée)

- Pas de « sachant que » ⇒ **pas d'auxiliaire** (hauteur, check_sum, path_sum : `s` est déjà dans l'énoncé).
- Il manque une info **d'au-dessus** (père, profondeur, reste…) ⇒ auxiliaire avec **un paramètre qui descend** (`prev`, `d`, `k`, `L`).
- Il faut **plusieurs infos d'en dessous** (nombre ET somme) ⇒ auxiliaire qui **renvoie un tuple**.
- Racine sans père : lui donner un faux père qui la laisse passer (ex. `B.key - 1`).

DFS 3 – Remonter un tuple (plusieurs infos, un seul parcours)

```
def __aux(B):
    """renvoie (ok, nb, somme) du sous-arbre B
       ok    : la règle est vraie partout
       nb    : nombre de noeuds
       somme : somme des clés"""
    (nb, s) = (1, B.key)        # B tout seul
    ok = True
    C = B.child
    while C and ok:             # stop dès que ok = False
        (ok, n, sc) = __aux(C) # infos du fils C
        ok = ok and TEST(n, sc, B)
                                # ex : sc / n <= B.key
        nb += n                 # + ses noeuds
        s += sc                 # + ses clés
        C = C.sibling
    return (ok, nb, s)

def f(B):
    return __aux(B)[0]          # on ne garde que ok
```

- Deux parcours séparés (`size` puis `somme`) = **points perdus**.
- La docstring de `__aux` est **obligatoire** : dire ce que vaut chaque case du tuple.
- La dernière fonction écrite = celle demandée (`f`).
- Exemples TD : `average_subtrees` → `(ok, nb, somme)`, `average_arity` → `(nb internes, somme arités)`.

DFS 4 – Un paramètre qui descend

Le tuple **remonte** des fils vers le père ; ici l'info **descend** du père vers les fils.

```
def __aux(B, d):
    """d = profondeur de B (racine : 0)"""
    if B.child is None:        # feuille :
        return d               # on renvoie sa profondeur
    s = 0
    C = B.child
    while C:
        s += __aux(C, d + 1)   # le fils est 1 cran
        C = C.sibling          # plus bas
    return s

def lce(B):
    return __aux(B, 0)         # on part de la racine
```

Exo	Param.	Vers le fils
cheminement	`d`	`d + 1`
clés niveau k	`k`	`k - 1`
chemin somme s	`s`	`s - B.key`
plus d'enfants	`nbc`	nb fils de `B`

Remplir une liste : la passer en paramètre et faire `L.append(...)`, sans la renvoyer.

Parcours parallèle `Tree` / `TreeAsBin`

```
def symmetric(T, B):
    if T.key != B.key:      # clés différentes : perdu
        return False
    C = B.child             # B lu de gauche à droite
    i = T.nbchildren - 1    # T lu de droite à gauche
    while C and i >= 0 and \
          symmetric(T.children[i], C):
        i -= 1              # on avance des deux côtés
        C = C.sibling
    return C is None and i == -1
                            # les deux finis ensemble
```

- Un **indice** avance dans `children`, un **pointeur** avance sur les frères.
- Fin : **les deux épuisés en même temps**.
- Tree → TreeAsBin : parcourir les fils **de droite à gauche** et chaîner chaque copie devant la précédente (`C.sibling = first`).

BFS – Par niveaux (marqueur `None` ou 2 files)

La file mélange deux niveaux. On repère la **fin d'un niveau** : c'est **le seul endroit** où on compare et où on s'arrête.

Marqueur `None`

```
q = Queue()
q.enqueue(T)
q.enqueue(None)          # None = fin du niveau 0
cur = 0                  # noeuds vus dans ce niveau
while not q.isempty():
    T = q.dequeue()
    if T is None:        # >>> FIN D'UN NIVEAU <<<
        ...              # ici : comparer, return...
        cur = 0          # on repart à 0
        if not q.isempty():  # sinon boucle infinie
            q.enqueue(None)  # fin du niveau suivant
    else:
        cur += 1         # un noeud de plus
        for C in T.children:
            q.enqueue(C) # ses fils : niveau suivant
```

Deux files

```
q = Queue()
q.enqueue(T)
while not q.isempty():      # 1 tour = 1 niveau
    q_next = Queue()        # file du niveau suivant
    cur = 0                 # noeuds vus dans ce niveau
    while not q.isempty(): # 1 tour = 1 noeud
        T = q.dequeue()
        cur += 1
        for C in T.children:
            q_next.enqueue(C)
    ...                     # >>> FIN DU NIVEAU <<<
                            # ici : comparer, return...
    q = q_next              # on passe au suivant
```

En `TreeAsBin`, remplacer le `for` par :
`C = T.child ; while C: q.enqueue(C) ; C = C.sibling`

Exo	Gardé d'un niveau à l'autre	À la fin de chaque niveau
largeur	`w`	`w = max(w, cur)`
level_up	`prev`	`if cur <= prev: return False` puis `prev = cur`
max par niveau	liste `L`	`L.append(m)`
niveau de x	`lvl`	`lvl += 1` (et `return lvl` dès que x sort)
niveaux croissants	`last`	`last = None`

Attention : `Queue` n'a pas de `len` → on compte avec une variable.

Tes pièges (vus en séance)

Piège	Faux	Juste
Nœud ou liste de fils	`f(T.children)`	`for C in T.children: f(C)`
Tester une feuille	`T.children == 0`	`T.nbchildren == 0`
	`B.child == 0`	`B.child is None`
Nœud comparé à un nombre	`if C == 0`	`if C.key == 0`
`return` qui coupe la boucle	`for ...: return 1 + f(C)`	accumuler, `return` après
Mauvais départ	`s = 1` pour les feuilles	« le nœud courant compte-t-il ? »
Frère oublié	`while C:` sans `C.sibling`	toujours `C = C.sibling`
Taille d'une file	`len(q)`	un compteur
Comparer trop tôt (BFS)	test dans le `for`	test à la fin du niveau

Checklist optimisation avant de rendre

- ☐ Arrêt dès que la réponse est connue
- ☐ Un seul parcours (tuple si besoin)
- ☐ Pas d'appel en double : `max(h, f(C))`
- ☐ Pas de liste construite pour rien
- ☐ Pas de `if` que le cas général gère
- ☐ Seulement `len range min max abs Queue`
- ☐ Fonctions auxiliaires documentées
- ☐ Dernière fonction = celle demandée